

Formally verifying consistency of sequence diagrams for safety critical systems

Xiaohong Chen^a, Qianqian Liu^a, Frédéric Mallet^{b,*}, Qin Li^a, Shubin Cai^{c,*}, Zhi Jin^d

^a Shanghai Key Laboratory of Trustworthy Computing, East China Normal University, Shanghai, China

^b Université Côte d'Azur, CNRS, Inria, I3S, France

^c College of Computer Science and Software Engineering, Shenzhen University, China

^d School of Computer Science, Peking University, Beijing, China

ARTICLE INFO

Article history:

Received 21 March 2021

Received in revised form 12 November 2021

Accepted 16 January 2022

Available online 25 January 2022

Keywords:

Safety critical systems

Sequence diagram

Behavior models

Clock graph

Formal verification

ABSTRACT

UML interactions, aka sequence diagrams, are frequently used by engineers to describe expected scenarios of good or bad behaviors of systems under design, as they provide allegedly a simple enough syntax to express a quite large variety of behaviors. This paper uses them to express safety requirements for safety critical systems in an incremental way, where the scenarios are progressively refined after checking the consistency of the requirements. The semantics of these scenarios are expressed by transforming them into an intermediate semantic model amenable to formal verification. In this paper, we rely on the Clock Constraint Specification Language (CCSL) as the intermediate semantic language. In some sense, sequence diagrams and CCSL constraints both express a family of acceptable infinite traces that must include the behaviors given by the finite set of finite execution traces against which we validate. We compare these requirements to actual execution traces to prove the validity of our transformation. As to check the consistency of the sequence diagrams, we present two verification methods based on SMT and clock graphs respectively. The SMT based method relies on our analysis tool called MyCCSL to analyze CCSL constraints. The clock graph based method transforms CCSL constraints into a clock graph, and does the analysis through traversing the clock graph. Finally, these two methods are evaluated against real cases from the railway transit systems. The results show that the SMT based method provides accurate but slow analysis, while the clock graph based method dramatically increases the verification efficiency aiming at two kinds of typical inconsistencies found by the SMT based method.

© 2022 Elsevier B.V. All rights reserved.

1. Introduction

The elicitation and formalization of safety requirements are very important steps in today's development processes of safety critical systems [1], especially when they have to follow safety critical software development standards such as railway application safety standards EN50128 and EN50129 [2]. In these standards, formal methods are strongly recommended to ensure safety because they could provide precise analysis.

* Corresponding authors.

E-mail addresses: xhchen@sei.ecnu.edu.cn (X. Chen), Frederic.Mallet@inria.fr (F. Mallet), shubin@szu.edu.cn (S. Cai).

But formal languages are not easy to learn and use. As a trade-off, engineers start to use semi-formal language such as UML (Unified Modeling Language) to specify requirements. As a kind of behavioral diagram, UML sequence diagrams provide allegedly a simple enough syntax to express a quite large variety of behaviors, including expected scenarios of good or bad behaviors of systems under design. So it is natural to use sequence diagrams to describe safety requirements. By safety requirements, we mean requirements that describe what actions and/or constraints should or should not be performed to maintain the system in a safe state. Different from other requirements, safety requirements not only describe what the system should do, but also what it should not do [3].

However, sequence diagrams cannot be analyzed and verified directly. There are many researches formalizing them. They range from early PROMELA-based model [4], timed automata network [5], calculus of communicating system [6] to various petri nets, including normal petri nets [7], colored Petri nets [8], Queueing Petri Nets [9], and Generalized Stochastic Petri Nets [10]. But they do not deal with negative operator which is necessary for safety requirements. Moreover, as requirements are usually provided in an incremental way, where the scenarios are progressively refined, consistency checking is essential. As some of their requirements are expressed in LTL or TCTL, it is difficult to check the consistency of requirements.

In this paper, we propose to provide automated consistency verification for sequence diagrams. As in many approaches before, the semantics of these scenarios are expressed by transforming them into an intermediate semantic model amenable to formal verification. We rely on the Clock Constraint Specification Language (CCSL) [11] as the intermediate semantic language. As a companion language of MARTE (Modeling and Analysis Real-Time Embedded Systems) [12], a profile of UML, CCSL describes the causal and temporal relations among events in terms of logic clocks. It is direct to describe “after ...it should do”, or “it should not do...” in terms of CCSL constraints. In order to prove the correctness of our transformation, we simulate the behaviors of sequence diagrams, and compare with the simulating behaviors of corresponding CCSL constraints according to the CCSL semantics.

Based on the CCSL constraints, we present two methods for consistency verification by extending our TASE 2020 paper [13]. The first is SMT based verification. It uses MyCCSL, an SMT based CCSL constraints analysis tool [14] to support the consistency checking. It is a heavyweight method. To improve the verification efficiency, this paper further analyzes the typical kinds of inconsistencies found by SMT based method, and presents a clock graph based method. It transforms the CCSL constraints into a clock graph, and then deduces the verification problem into a graph search problem. This is a lightweight method. To sum up, we make the following contributions:

- Sequence diagrams are transformed to CCSL, and verified against consistency using SMT based analysis tool automatically, which makes sequence diagrams suitable for safety critical systems.
- A clock graph is defined and a clock graph based verification of sequence diagrams is presented, which dramatically improve the efficiency of SMT based method.

Finally, these two methods are evaluated using real cases from railway transit systems. The results show that SMT based verification provides accurate but slow analysis, while the clock graph based verification dramatically increases the verification efficiency by only aiming at two kinds of typical inconsistencies.

The remainder of the paper is organized as follows. Section 2 introduces CCSL, MyCCSL and a running example. Section 3 defines the SMT based verification method. Section 4 presents the clock graph based verification method. Section 5 evaluates our work. Section 6 compares related work, and finally Section 7 concludes.

2. Preliminaries

This section introduces the Clock Constraint Specification Language (CCSL), its analysis tool MyCCSL, and a running example for better understanding.

CCSL [11] provides descriptions of the causal and temporal relations among events in terms of logic clocks. The related concepts include *clock*, *instant*, *instant relation* and *clock constraint*. Each event could be a logical clock, which models the access to time. According to [15], a *clock* is defined as a sequence of event occurrences. Each occurrence of the event, is called an *instant*, which is also called a tick of clock. A clock is formally defined as a pair tuple, i.e., $C := \langle I, < \rangle$ where, I is a set of discrete instants, $<$, named strict precedence, is a quasi-order relation on I . $C[k]$ denotes the k_{th} instant in I , and $k \in \mathbb{N}_{>0}$ is called the index.

Instants in clocks have relations, including *Precedence*, *Strict Precedence*, and *Coincidence*. Precedence ($\preceq$) represents causal dependency relation between instants. Coincidence ($\equiv := \preceq \cap \succeq$) is a strong relation that forces simultaneous occurrences of instants. Strict Precedence ($< := \preceq \setminus \equiv$) represents the sequential relations of instants.

The clock constraints could be defined by millions of *instant constraints*. There are two kinds of clock constraints, clock relations and clock expressions. Clock relations such as *StrictPre*($<$), *Alternate*($\sim$), *Coincidence*($=$), *SubClock*($\subseteq$) and *Exclude*($\#$), specify the relation between clocks. Clock expressions such as *DelayFor*($\$$) and *Union*($+$), define new clocks and provide rules for creating new clocks based on existing clocks. Table 1 gives definitions of clock constraints used in this paper and provides their intuitive meanings.

MyCCSL is a tool for analyzing CCSL constraints [14]. Its main function is to translate the CCSL constraints into SMT formula, and provide various analysis, such as trace analysis, deadlock detection, LTL model checking, and schedulability analysis. The most effective SMT solver Z3 [16] is integrated in. As the SMT could effectively mitigate state explosion

Table 1
Clock constraints used in this paper.

Clock constraint	Semantic description	Description
C_1 <i>StrictPre</i> C_2 ($C_1 < C_2$)	$\forall i \in N_{>0}, C_1[i] < C_2[i]$	restricts that the i_{th} instant of C_1 ticks strictly before the i_{th} instant of C_2 .
C_1 <i>Alternate</i> C_2 ($C_1 \sim C_2$)	$\forall i \in N_{>0}, C_1[i] < C_2[i] \wedge C_2[i] < C_1[i+1]$	restricts an alternation of ticks of the left and right clocks.
C_1 <i>Coincidence</i> C_2 ($C_1 = C_2$)	$\forall i \in N_{>0}, C_1[i] \equiv C_2[i]$	restricts that the i_{th} instant of C_1 and the i_{th} instant of C_2 tick at the same time.
C_1 <i>SubClock</i> C_2 ($C_1 \subseteq C_2$)	$\forall i \in N_{>0}, \exists j \in N_{>0}, C_2[j] \equiv C_1[i]$	restricts that C_1 occurs at some step only if C_2 occur at this step as well.
C_1 <i>Exclude</i> C_2 ($C_1 \# C_2$)	$\forall i, j \in N_{>0}, C_1[i] \neq C_2[j]$	restricts that two clocks never tick together.
$C_1 = C_2$ <i>DelayFor</i> d ($C_1 = C_2 \$ d$)	$\forall i \in N_{>0}, C_1[i] \equiv C_2[i+d]$	defines a clock C_1 which ticks together with C_2 after C_2 ticks d times.
$C_1 = C_2$ <i>Union</i> C_3 ($C_1 = C_2 + C_3$)	$\forall i \in N_{>0}, \exists j, k \in N_{>0}, (C_1[i] \equiv C_2[j]) \vee (C_1[i] \equiv C_3[k])$	defines a clock C_1 which ticks iff C_2 or C_3 ticks.

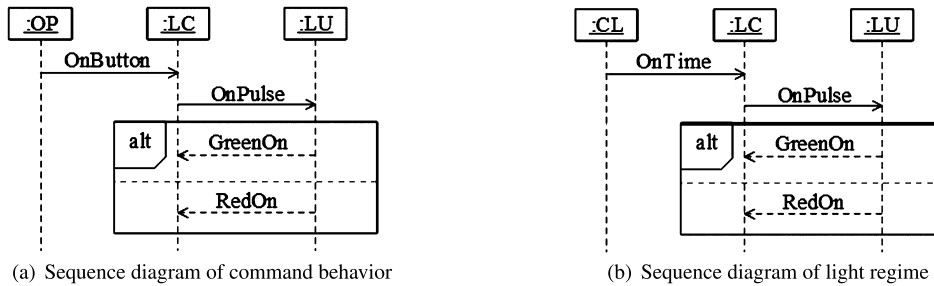


Fig. 1. Sequence diagrams of the light controller system.

problem in model checking, MyCCSL is very effective. In this paper, we will use schedulability analysis to support consistency checking of safety requirements. In the schedulability, if all the constraints are schedulable, MyCCSL returns *sat* and a schedule. Else, *unsat* is returned with a counterexample which means that all clocks can not tick under current clock constraints.

Running example We provide a simple lighting control system as a running example. The system is to control a set of *Light Unit (LU)* according to the *Operator (OP)*'s command or time. Fig. 1 shows two sequence diagrams of these two control ways. In Fig. 1(a), the *OP* presses *OnButton* (*OnButton*) which can trigger *OnPulse*. The state of *LU* can automatically be green or red. In Fig. 1(b), the *timer Clock (CL)* can also trigger the *OnPulse*.

3. SMT based verification

In this section, we introduce the process of SMT based verification, including transformation of sequence diagrams to CCSL constraints, proof of the transformation, and checking with the tool MyCCSL.

3.1. Transformation from sequence diagram to CCSL constraints

As the definitions and semantics of UML 2 sequence diagrams are rather informal and ambiguous [17], we facilitate the automated creation of the CCSL constraints by defining transformation rules for formally representing constructs of sequence diagrams. The main objective is to represent the sequence diagram's constructs and their relations in a CCSL formalism for further consistency verification.

In the sequence diagram, there are many object lifelines. A lifeline has many interactions. Interactions are messages along with sending and receiving OccurrenceSpecifications (*OSs*) covered by lifelines in temporal order, associated CombinedFragments included operands. An interaction e is called “receiving event” of interaction i “sending event” if there is a link from i to e that synchronizes with the appropriate sending and receiving lifelines.

In our formalization, we view a message and its sending and receiving *OSs* (i.e., events) as a tuple $\langle Lf, msg, snd/rec \rangle$, where Lf represents the lifeline that is responsible for sending or receiving messages, msg denotes the message name, snd and rec represent the sending *OS* and receiving *OS* of the corresponding message on a lifeline, respectively. We map this tuple into **two logical clocks**, snd and rec . In order to more precisely describe the snd and rec , we name them Lf_msg_snd and Lf_msg_rec .

For each sequence diagram mapping, we first find all the lifelines, then the sending and receiving events, and finally the causal and temporal relations among these events/clocks. According to [17], transformation rules are made by considering

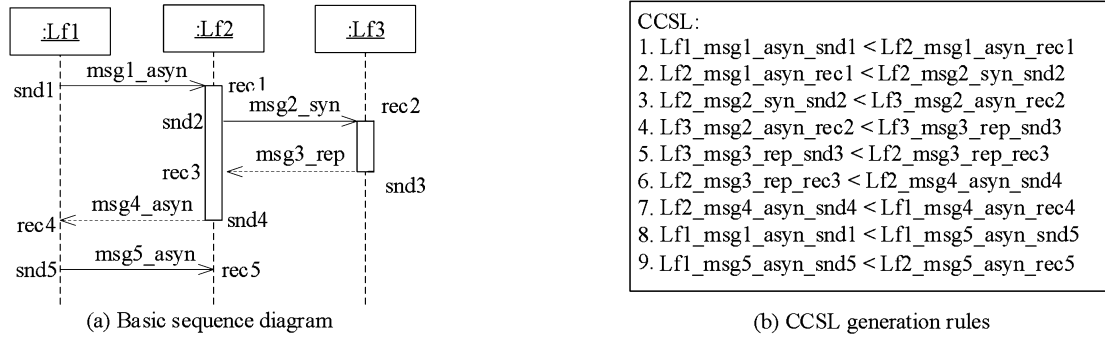


Fig. 2. Transformation of basic sequence diagram.

the following situations. Due to limited space, the mapping rules for Option, Break, Ignore and Consider combined fragments into CCSL are omitted.

3.1.1. Basic sequence diagram

A sequence diagram without a CombinedFragment is referred to as a basic sequence diagram (such as the one in Fig. 2(a)). The following rules for sending and receiving messages must be considered as the semantics of a basic sequence diagram suggests.

- The OSs on the same lifeline must occur in the same order in which they are described [8, p.505], which means they keep the *StrictPre* relations on the same lifeline.
- “The semantics of a complete message is simply the trace *sendEvent*, *receiveEvent*” [8, p.507]. A receiving OS of a message is enabled for execution if and only if the sending occurrence of the same message has already occurred [8, p.507]. This means these events have the *StrictPre* relation: *sendEvent* < *receiveEvent*.
- If sending and receiving OSs of the same message are on the same lifeline then the sending event of a message must exist before its receiving event [8, p.506]. This also keeps the *StrictPre* relations between them.
- For the synchronized message *syn_msg*, only after the response *msg_rep*, the next message *nmsg* could be sent. So we get: *syn_msg_snd* < *msg_rep* and *msg_rep* < *nmsg_snd*.
- For the asynchronous message, *asyn_msg*, no matter whether response *msg_rep* is received or not, the next message *nmsg* could be sent, i.e., *syn_msg_snd* < *msg_rep*, and *syn_msg_snd* < *nmsg_snd*.

As the *strictPre* relations are transitive, it is unnecessary to write all them down. We use the depth first search algorithm to find all the messages and transform them. Fig. 2(a) shows an example of the basic sequence diagram. *Msg1_async* is a message sent from lifeline *Lf1* to the lifeline *Lf2*. The particular sent/receiving rule is shown in Fig. 2(b) line 1. *Msg2_syn* is a synchronized message, with *msg3_rep* being its response. It means only after receiving OS (rec3) is enabled, the next message *msg4_async* could occur. They can be transformed into CCSL constraints shown in Fig. 2(b) line 4-7. *Msg1_async* is also an asynchronous message, with *msg4_async_rec* being its response, and *msg5_async* being its next message. It means that there is no need to wait for *msg4_async_rec4*, *msg5_async* can be sent after *Msg1_async*. The transformed CCSL constraints are in Fig. 2(b) line 8-9.

3.1.2. Weak sequencing combined fragment

The **seq** interaction operator imposes the order of the execution of operands associated with the same lifeline with the following constraints [8, p.483]:

- The ordering of events (i.e., OSs) within each of the operands is maintained.
- OSs on different lifelines from different operands may execute in any order.
- OSs on the same lifeline from different operands are ordered so that an OS of the first operand comes before that of the second operand.

Fig. 3(a) shows an example of a Weak Sequencing combined fragment with the same lifelines from different operands, and Fig. 3(b) illustrates the rules for mapping it into CCSL constraints. In operand 1, message *msg1* and *msg2* keep the same rules as in the basic sequence diagram as shown in line 1-4 of Fig. 3(b). In operand 2, message *msg3* shares the same lifelines with *msg1* and *msg2*. For *Lf1*, the OS *snd2* must happen before *snd3*, while for *Lf2*, the OS *rec2* must happen before *rec3*. So we get rules of *msg3* in line 5-7 of Fig. 3(b).

Fig. 3(c) is another example of a Weak Sequencing combined fragment with different lifelines from different operands, and Fig. 3(d) illustrates the rules for mapping it into CCSL constraints. In operand 1, message *msg1* and *msg2* keep the same

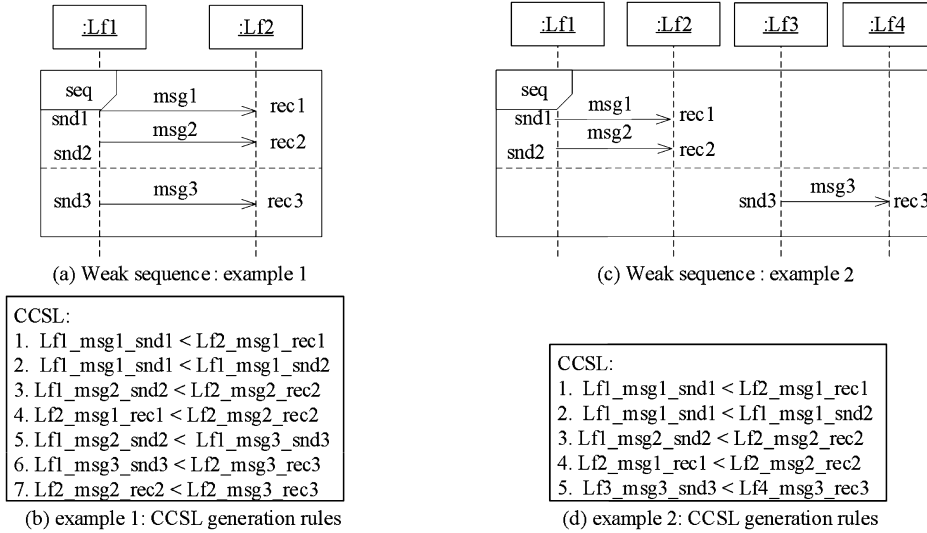


Fig. 3. Transformation of Weak Sequencing combined fragment.

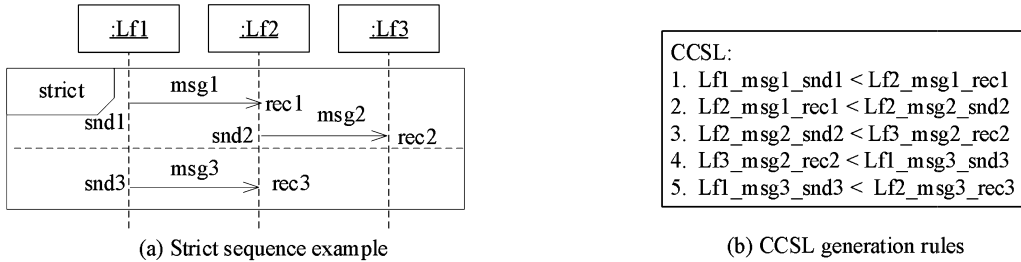


Fig. 4. Transformation of Strict Sequencing combined fragment.

rules as in the basic sequence diagram as shown in line 1-4 of Fig. 3(d). In operand 2, *msg3* has a different lifeline with *msg1* and *msg2*. The occurrence of *msg3* is independent from *msg1* and *msg2*. So we only get a rule in line 5 of Fig. 3(d).

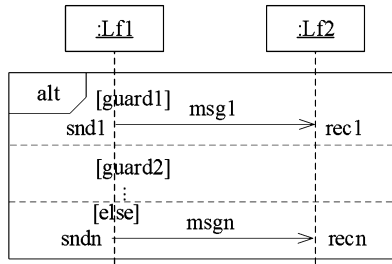
3.1.3. Strict sequencing combined fragment

The semantics of **Strict** Sequencing (i.e., strict interaction operator) imposes the total order between adjacent operands. It contains a stronger version of the second rule introduced for Weak Sequencing, in particular, *OSs* on different lifelines from different operands have strict order of execution [8, p.483]. In other words, the first *OS* in a succeeding operand cannot be enabled until all the *OSs* on all the covered lifelines within the preceding operand have completed. Any covered lifeline needs to wait for other lifelines to enter the second or subsequent operand. For instance, sending *OS* of a message *msg3* within the second operand covered by a lifeline *Lf1* will not be executed until the last *OS* that is *rec2* within a first operand on a lifeline *Lf3* finishes its execution, as shown in Fig. 4(b) (Line 1-4). Fig. 4(b) shows its CSDL generation rules.

3.1.4. Alternative combined fragment

In the UML 2 specification [8, p.482], an alternative combined fragment describes a branching operation in a sequence diagram. The **alt** operator of the combined fragment represents a choice of behavior where at most one of the operands will be selected whose interaction constraint (guard condition) evaluates to True (i.e., an if-then-else statement). The else guard is the negation of the disjunction of all other constraints in the enclosing combined fragment. If none of the operands has a guard that evaluates to True, none of the operands will be executed and the remainder of the enclosing InteractionFragment will be performed. This also means, only one of the *OS* in these operands occurs. They are exclusive.

Fig. 5(a) shows an example of an alternative combined fragment, whose guard is encoded as a boolean variable. If the guard of the first operand evaluates to true, the *OSs* enclosed within the first operand are executed, otherwise the whole operand is skipped. In this operand, the messages hold the temporal relations (line 1-4 in Fig. 5(b)). The choice among alternatives is made using *Union* and *exclude* relations in Fig. 5(b) (Line 5-6). To ensure the corresponding *rec* of *snd* is sure to occur, we union all the *recs* and use *Alternate* as shown in Fig. 5(b) (Line 7-8).



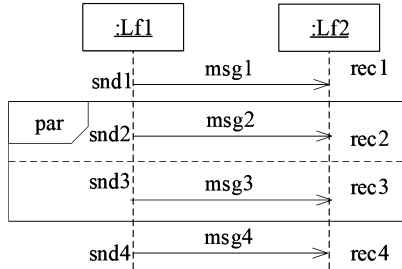
(a) Alternatives example

CCSL:

1. $Lf1_msg1_snd1 < Lf2_msg1_rec1$
2. $Lf1_msg2_snd2 < Lf2_msg2_rec2$
3. ...
4. $Lf1_msgn_sndn < Lf2_msgn_recn$
5. $begin = Lf1_msg1_snd1 + Lf1_msg2_snd2 + \dots + Lf1_msgn_sndn$
6. $Lf1_msg1_snd1 \# Lf1_msg2_snd2 \# \dots \# Lf1_msgn_sndn$
7. $end = Lf2_msg1_rec1 + Lf2_msg2_rec2 + \dots + Lf2_msgn_recn$
8. $begin \sim end$

(b) CCSL generation rules

Fig. 5. Transformation of Alternative combined fragment.



(a) Parallel fragment

CCSL:

1. $Lf1_msg1_snd1 < Lf2_msg1_rec1$
2. $begin = Lf1_msg2_snd2 + Lf1_msg3_snd3$
3. $Lf2_msg1_rec1 < begin$
4. $Lf1_msg2_snd2 < Lf2_msg2_rec2$
5. $Lf1_msg3_snd3 < Lf2_msg3_rec3$
6. $end = Lf2_msg2_rec2 + Lf2_msg3_rec3$
7. $end < Lf1_msg4_snd4$
8. $Lf1_msg4_snd4 < Lf2_msg4_rec4$

(b) CCSL generation rules

Fig. 6. Transformation of Parallel combined fragment.

3.1.5. Parallel combined fragment

A Parallel combined fragment is denoted by an interaction operator **par** which defines potentially parallel execution of behaviors of the operands [8, p.483]. The OSs of different operands can be interleaved in any way as long as the ordering imposed by each operand is preserved. In other words, OSs of messages within the same operand respect the order along a lifeline whilst OSs of messages on the same lifeline from different operands are ordered so that the first message occurrence of the operands has the same preceding OS.

Fig. 6(a) shows an example of parallel combined fragment, and Fig. 6(b) shows its transformation into CCSL constraints. In this figure, a sending OS of a message *msg1* ($Lf1_msg1_snd1$) on a lifeline *Lf1* leads to the execution of sending OSs of messages in both operands (i.e., $Lf1_msg2_snd2$ and $Lf1_msg3_snd3$). It is translated to line 1-3. Similarly, the receiving OS of messages in both operands, (i.e., $Lf2_msg2_rec2$ and $Lf2_msg3_rec3$), trigger *msg4*, is transformed into CCSL constraints in Line 6-8 of Fig. 6(b).

3.1.6. Loop combined fragment

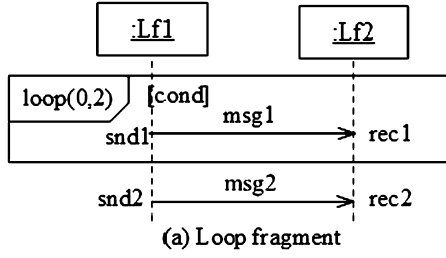
In Loop combined fragments, the interaction operator **loop** defines that its sole operand will be repeated for at least the minimum (minint) number of times and at most maximum (maxint) number of times as long as the guard condition remains True [8, p.485]. If the loop has no bounds, this means that an indefinite loop (with minint = 0 and maxint = infinite) is executed. However, it is unrealistic for most loops that they really execute indefinitely, and therefore, we assume that loops will eventually stop.

Fig. 7(a) shows an example of a Loop combined fragment having minint = 0 and maxint = 2. To deal with Loop combined fragments, we use a counter to record the execute number. Set it to 0 at first. After the end of the current iteration, the counter is increased by one at the beginning of the next iteration. Furthermore, the loop condition and counter are checked at the beginning of each iteration. If the condition is evaluated to false or counter is greater or equals to maxint, a new iteration cannot start and execution of the loop will terminate. The OSs of all the messages within the operand among iterations execute sequentially along a lifeline. For example, the sending and the receiving of the message *msg1* are executed twice. We create a virtual message *temp*. It will happen when the second receiving of message *msg1* happens. So $temp = msg1_rec\$2$. And definitely *temp* happens before the sending OS (*snd2*). Fig. 7(b) shows its CCSL constraints.

3.1.7. Negative combined fragment

The interaction Operator **neg** designates traces that are defined to be invalid [8, p.482]. In other words, the negation operator is used to denote the case of some not-executing events, or forbidden behavior. Therefore, it is particularly suitable for describing safety requirements.

Fig. 8(a) shows an example of negative combined fragment, and 8(b) shows the corresponding CCSL constraints. In this figure, a sending OS of a message *msg2* ($Lf1_msg2_snd2$) on the lifeline *Lf1* is forbidden to happen between *msg1* and

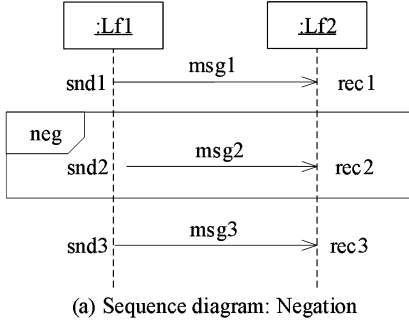


CCSL:

1. $Lf1_msg1_snd1 < Lf2_msg1_rec1$
2. $temp = Lf2_msg1_rec1 \$2$
4. $temp < Lf1_msg2_snd2$
5. $Lf1_msg2_snd2 < Lf2_msg2_rec2$

(b) CCSL generation rules

Fig. 7. Transformation of Loop combined fragment.



CCSL:

1. $Lf1_msg1_snd1 < Lf2_msg1_rec1$
2. $Lf1_msg2_snd2 < Lf2_msg2_rec2$
3. $Lf1_msg3_snd3 < Lf2_msg3_rec3$
4. $Lf1_msg1_snd1 \sim Lf1_msg3_snd3$
5. $Lf1_X_send < Lf1_msg1_snd1 \1
6. $Lf2_X_rec < Lf2_msg1_rec1 \1
7. $Lf1_X_snd < Lf2_X_rec$
8. $Lf2_msg3_rec3 < Lf1_X_snd$
9. $Lf1_msg2_snd2 \subseteq Lf1_X_snd$
10. $Lf2_msg2_rec2 \subseteq Lf2_X_rec$

(b) CCSL generation rules

Fig. 8. Transformation of Negative fragment.

$msg3$. So it is only allowed to occur between $msg3$ and next time $msg1$. We create a virtual message X . It happens after $msg3$, but before next $msg1$ (Line 5-8 in Fig. 8(b)). $msg2_snd$ is a *SubClock* of X_snd , and $msg2_rec$ is a *SubClock* of X_rec (Line 9-10 in Fig. 8(b)). This forces $msg2$ either happens together with X or not happen.

3.2. Proof of transformation

In this section, we analyze typical sequence diagrams, and try to find the relationship between the sequence diagram semantics and CCSL constraint semantics. In CCSL, Yin et al. [18] propose an operational semantics for the CCSL constraints in terms of finite state machines (FSMs). So in this paper, we also figure out the sequence diagram semantics in FSMs, and compare these two kinds of FSMs accordingly.

Basic sequence diagram For basic sequence diagram, both weak and strict sequencing combined fragments can be transformed into *StrictPre* relations. We only consider one message in a sequence diagram as shown in Fig. 9(a). From the execution of sequence diagram, we could get an FSM as shown in Fig. 9(b). It means that the execution trace is $< snd1, rec1 >$.

The corresponding CCSL constraints are shown in Fig. 9(c). According to the *StrictPre* semantics in [18], we get the FSM of this constraint. The index difference between $snd1$ and $rec1$ is used to express a state. Since *StrictPre* can have infinite states, we bound the difference to be 1, which means that $snd1$ happens at most one tick before $rec1$. Reviewing it in the *StrictPre* operational semantic, an FSM is obtained in Fig. 9(d). The meaning is as follows. At first, the index difference is 0 in state S_0 . Then $snd1$ happens, index difference becomes 1, then it goes to state S_1 . From S_1 , either $snd1$ and $rec1$ happen together, or $rec1$ happens. If they happen together, the index difference is still 1, so it transits to S_1 . If $rec1$ happens, the index difference changes back to 0, so it jumps back to S_0 .

Alternative combined fragment For the alternatives, we use the simplest situation (Fig. 10(a)). Only two messages are given. From the sequence diagram semantics, an FSM is obtained in Fig. 10(b). It means either $snd1, rec1$ happens, or $snd2, rec2$ happens.

The corresponding CCSL constraints are shown in Fig. 10(c). According to the operational semantics of *StrictPre*, *Union*, *Alternate* and *Exclude* in [18], the corresponding FSM of these constraints are given in Fig. 10(d). The *Exclude* relation prohibits the simultaneous occurrences of $snd1$ and $snd2$ from state S_0 , and the *Alternate* relation forbids the simultaneous occurrences of $snd1$ and $rec1$ from state S_2 , and $snd2$ and $rec2$ from state S_3 .

Parallel combined fragment For the parallel combined fragment, we choose only two operands with two messages in Fig. 11(a). According to the sequence diagram semantics, $snd1, snd2, rec1, rec2$, could happen in any order only $snd1$ ($snd2$) must be before $rec1$ ($rec2$). So we get a FSM in Fig. 11(b).

The corresponding CCSL constraints are listed in Fig. 11(c). They are two *StrictPre* constraints. To control the complexity, we still bound the index difference to 1. For a *StrictPre* constraint, its FSM is shown in Fig. 9(d). Following the composition rule in [18], we compose the two semantics together, and then obtain the CCSL constraints' FSM in Fig. 11(d).

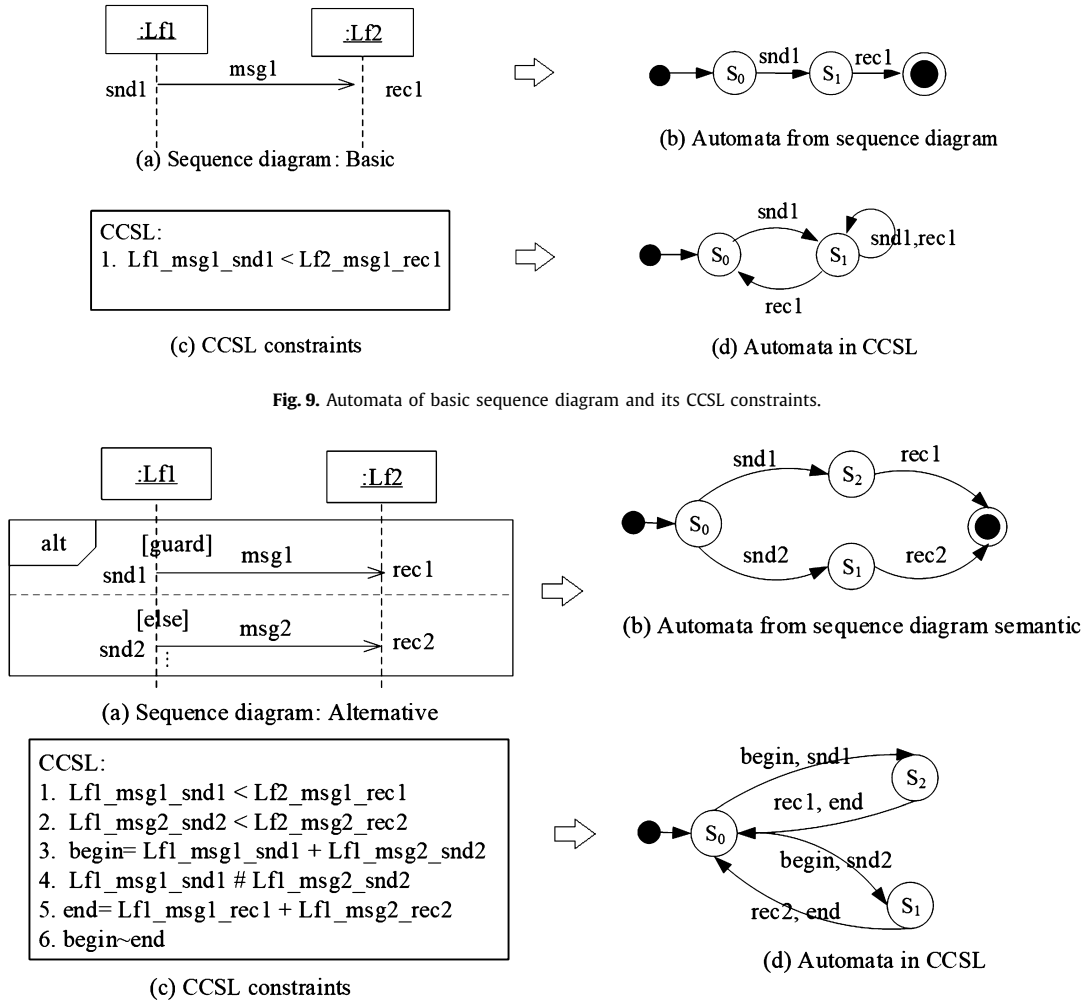


Fig. 9. Automata of basic sequence diagram and its CCSL constraints.

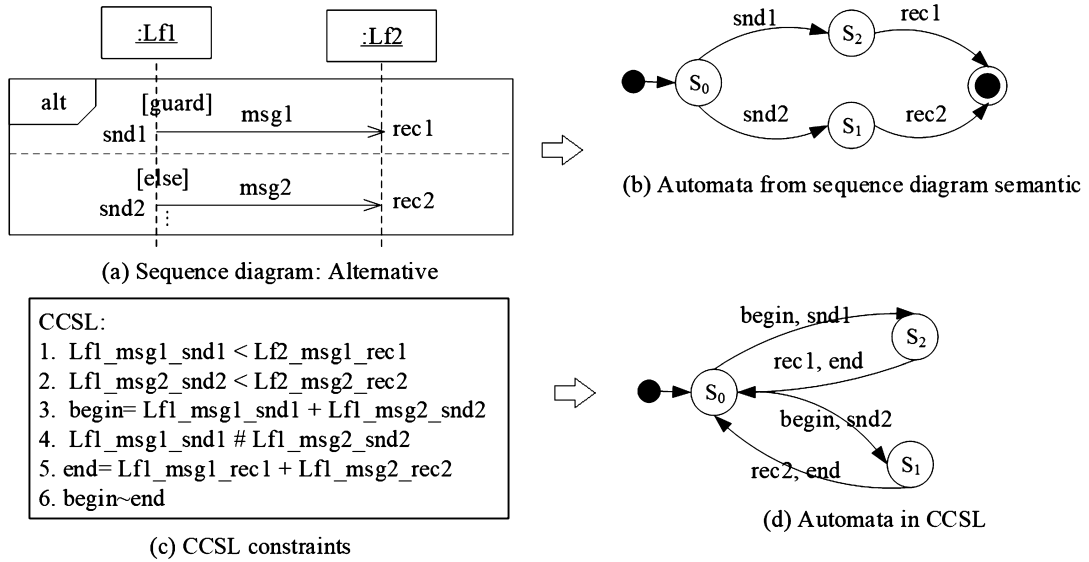


Fig. 10. Automata of alternative combined fragment and its CCSL constraints.

Loop combined fragment We choose a loop with one message as shown in Fig. 12(a). According to the sequence diagram semantics, *snd2* and *rec2* happen two times, while *snd1*, *rec1*, *snd3* and *rec3* could happen once. The trace is $\langle \text{snd1}, \text{rec1}, \text{snd2}, \text{rec2}, \text{snd2}, \text{rec2}, \text{snd3}, \text{rec3} \rangle$. So we get the FSM in Fig. 12(b).

The CCSL constraints of the sequence diagram are shown in Fig. 12(c). They include *Alternate*, *StrictPre*, and *DelayFor* constraints. We compose the operational semantics of the three kinds constraints together, then get an FSM as shown in Fig. 12(d). The *Alternate* relation forbids the simultaneous occurrences of *snd1*, *rec1*, *S2*, *snd2*, *rec2*, *snd3*, or *rec3* from any state. The temporary clock *temp* ticks with the second occurrence of *rec2*.

To sum up, in each figure (from Fig. 9 to Fig. 12), the automata in (d) could simulate (b), which means our CCSL semantics have larger state space. This shows that our transformation from sequence diagrams to CCSL constraints is not equivalent. But when doing the verification, if the safety requirements pass our consistency verification, it will surely satisfy the consistency requirements of sequence diagrams. That is enough for the verification.

3.3. Consistency property and MyCCSL based checking

The consistency property in sequence diagrams means that there are no event conflicts, i.e., for any two events e_1 and e_2 in one sequence diagram with a certain order between them, they keep the same sequence in another sequence diagram. Under this sense, the two events have different orders in different sequence diagrams will be inconsistent. Since we transform all the events and their relations to clocks and clock constraints in CCSL, we will check out event conflicts in CCSL. For a set of CCSL constraints, if there are clock conflicts, there will be no clock (any one of the conflicting clocks) ticks in a clock schedule. This can be done by the schedulability analysis by MyCCSL. MyCCSL is used to find out whether there is any clock ticks in a clock schedule. If there is no clock ticks, *unsat* is returned. Else, *sat* will be returned. So we deduce the consistency checking of sequence diagrams to be the schedulability analysis. The result *unsat* means that the sequence

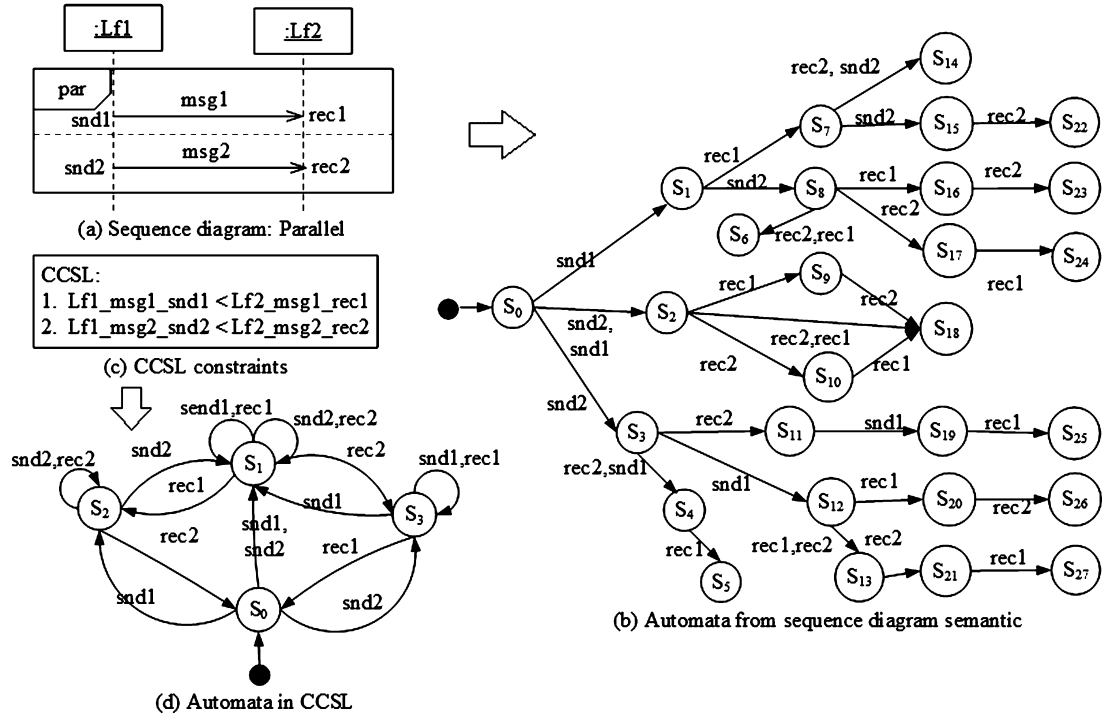


Fig. 11. Automata of parallel combined fragment and its CCSL constraints.

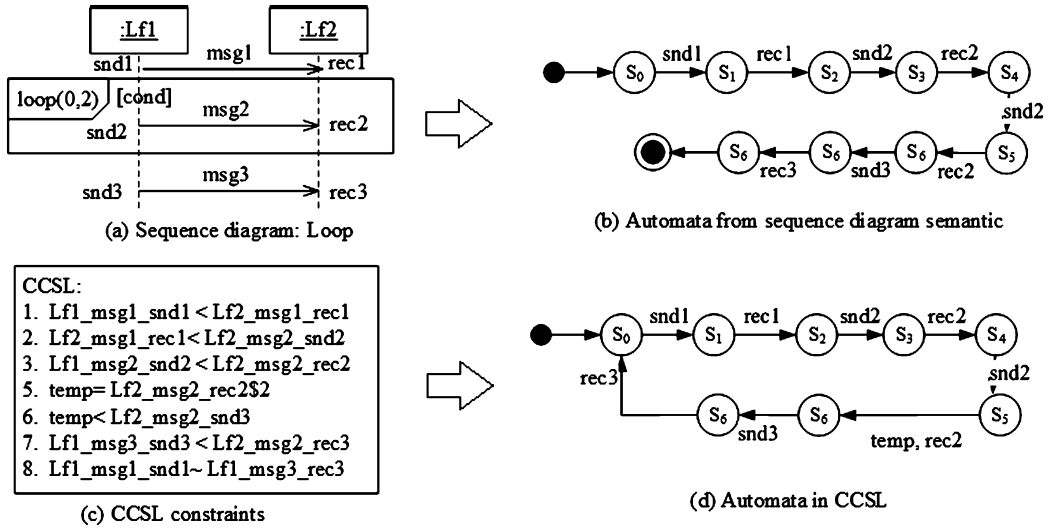


Fig. 12. Automata of loop combined fragment and its CCSL rules.

diagrams are inconsistent, while *sat* means consistency. Therefore, we only need to check whether these clock constraints are schedulable or not.

Before putting the CCSL constraints into MyCCSL, we compose the CCSL constraints together. The composition of CCSL constraints is not just putting them together. There are many temporary variables that may be shared by many constraints. We need to traverse all constraints that contain temporary variables and determine whether these notations are the same or not. The following two situations should be considered:

- There may be some temporary variables with different names but actually the same from different sequence diagrams. For example, if there is a constraint " $tmp_1 = C_1\$1$ " from a sequence diagram, and a constraint " $tmp_2 = C_1\$1$ " from another sequence diagram, it is found that the tmp_1 and tmp_2 describe the *DelayFor* relation of the same clock. In this situation, keep only one temporary variable and replace the name of the temporary variable in other CCSL constraints.

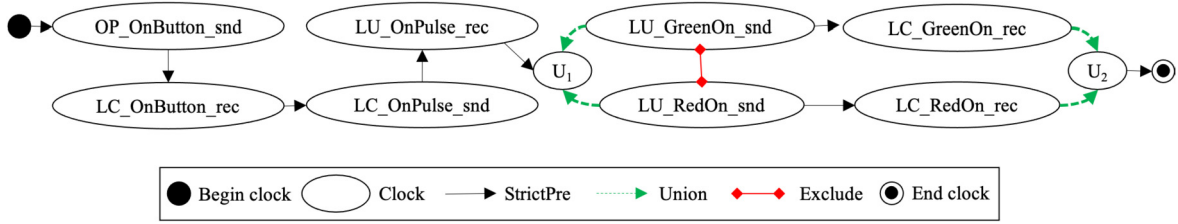


Fig. 13. The clock graph transformed from running example of Fig. 1(a).

- There may be some temporary variables with the same name, but in fact they are different. For example, if there is a constraint “ $tmp_1 = C_1\$1$ ” from one diagram, and a constraint “ $tmp_1 = C_2\$1$ ” from another diagram. Although they share the same name tmp_1 , they describe the *DelayFor* relation of different clocks. In this case, we create a new temporary variable tmp_2 to represent the *DelayFor* relation of the clock C_2 , and replace all the temporary variables in the constraints related to tmp_1 with tmp_2 in the second diagram.

After the composition, we put all the CCSL constraints directly into MyCCSL. In MyCCSL, the schedulability analysis is used to find whether there is any clock ticks in a schedule. If there is no clock ticks, *unsat* is returned. In our consistency checking, if the safety requirements are inconsistent, there must be no clock ticks. Therefore, we only need to check whether these clock constraints are schedulable or not. In other words, if *unsat* is returned in the schedulability analysis, the safety requirements are inconsistent.

As a model checker, MyCCSL only supports the core CCSL expressions. Not all the clock constraints used in this paper are defined in MyCCSL. Constraint *Alternate* needs to be transformed. The *Alternate* ($C_1 \sim C_2$), could be expressed using the *DelayFor*(\$), i.e., $C_1 = C_2\$1$. It means between $C_1[i]$ and next $C_1[i + 1]$, $C_2[i]$ could be inserted.

Before the verification, one needs to input the running parameter: the program timeout period to prevent the running time from being too long to terminate; the SMT solver can choose Z3 to solve the SMT formulae; the boundary value, 0 represents unbounded verification, usually we will enter a positive integer as the bound of verification; and whether this is periodic or not. Then one can click *Run*. It will give you a result *sat* or *unsat* in a short time.

4. Clock graph based verification

In the clock graph based verification, we try to visualize all the clock constraints by defining a clock graph, and use the graph search to find inconsistency. There are two main difficulties. The first one is how to transform all the clock constraints to causality dependency constraints like *StrictPre*. Among the 7 constraints in Table 1, only *Union* and *SubClock* cannot be addressed because they are not causality relations [19]. But they have temporal information, and could be transformed to *Coincidence* and *StrictPre* by decomposition. So we propose to decompose the clock graph after composing the original clock graphs together, and use the decomposition to remove these two kinds of constraints. Moreover, the decomposition could speed up the verification.

The second difficulty lies in how to define the inconsistency property in terms of clock graphs. We try to find the property from the SMT based verification results, and finally get two kinds of inconsistencies. In this section, we firstly define the concept of a clock graph, then compose the clock graphs that need verification. The clock graph is decomposed into many sub-clock graphs, and being searched for inconsistency with predefined properties.

4.1. Definition of a clock graph

According to [19], we present the clock graph as an intermediate structure for visualizing the CCSL specification. Each node is a clock, and each edge is a clock constraint. Moreover, to express paths explicitly in the graph, we represent the begin (B) and end (E) nodes in the graph.

Definition 1. A clock graph (CG) is a directed graph, $CG := (C, A)$, where,

- C is a set of nodes denoting clocks.
- $A \subseteq C \times R \times C$ is a set of labelled directed edges denoting the given clock constraints, where R is the set of relations among clocks. The relations among clocks in R are the 7 constraints introduced in Table 1.

More concretely, let $CG = (C, A)$ be a clock graph, $(C_1, r, C_2) \in A$ means that there exists constraint relation r from C_1 to C_2 . Among them, *StrictPre*, *Alternate*, *Coincidence*, *Exclude* and *DelayFor* are causality relations [19], while *SubClock* and *Union* are not. *DelayFor*(d) is a parameterized relation with parameter d and *Exclude* is a bi-directed relation. The *Union* relation relates one clock to two clocks, implying that there exist two directed edges from the source node to the sink nodes.

Fig. 13 gives a CG of the running example in Fig. 1(a). It contains three types of constraints:

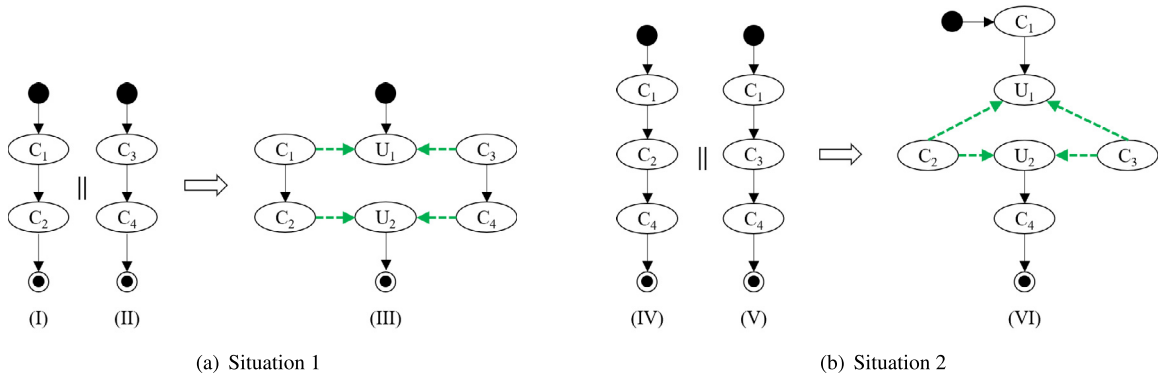


Fig. 14. Schematic diagram of clock graph composition situations.

- *StrictPre*. e.g., $CL_OnButton_snd < LC_OnButton_rec$;
- *Exclude*. e.g., $LU_RedOn_snd \# LU_GreenOn_snd$;
- *Union*. e.g., $Begin = LU_RedOn_snd + LU_GreenOn_snd$.

4.2. Clock graph composition

Clock graph composition is to add the parallel relations among clock graphs that transformed from different sequence diagrams. The parallel relation is in fact the *Union* constraint in CCSL. By adding it, the sequences between clocks of each clock graph will not change. Two situations are under consideration:

- **Situation 1**: there is no shared clock in to-be composed graphs except the begin and end nodes. We only need to create two virtual clocks to connect the begin and end clocks of different graphs. For example, if one clock graph has two clocks C_1 and C_2 and the constraint $C_1 < C_2$, another clock graph has two clocks C_3 and C_4 and the constraint $C_3 < C_4$. To ensure the sequences between them, we create a virtual U_1 clock to union C_1 and C_3 , and an U_2 clock to union C_2 and C_4 . The clock graph (I) and (II) are composed into clock graph (III) as shown in Fig. 14(a);
- **Situation 2**: there are shared clocks in to-be composed graphs. For example, the left side of Fig. 14(b) shows two clock graphs. We can find that the two clock graphs have the same clocks C_1 and C_4 , so we create two virtual clocks U_1 and U_2 for two different clocks to ensure that the order between clocks remains unchanged. The clock graph (IV) and (V) are composed into clock graph (VI) as shown in Fig. 14(b).

According to the above analysis, we design a composition algorithm in terms of clock graphs. The inputs are two clock graphs, and the output is a composed clock graph. First, we directly compose the sets of nodes and edges together as the initial composition result. Then shared nodes are searched in the input clock graphs with their precursors and successors. Accordingly, we delete the original edges between the shared node and precursors (successors), create a union node, and add the constraints between the union node and the precursors or successors as well as the *strictPre* relation between shared nodes and the union nodes. The specific process is given in Algorithm 1. The complexity of this algorithm is $O(n^2)$, where n is the edge number of a clock graph.

We use the running example to illustrate the composition algorithm. Firstly, we compose all edges of two clock graphs corresponding to the sequence diagram in Fig. 1 and remove duplicate edges. Then it is found that the two clock graphs have different begin nodes. A virtual clock U_3 is created to union $OP_OnButton_snd$ and CL_OnTime_snd as well as corresponding edges. Similarly, a virtual clock U_4 is created to union $LC_OnButton_rec$ and LC_OnTime_rec . Moreover, three edges are added. Two represent the *Union* relation and one represents the *StrictPre* relation between the virtual U_4 clock and its successor clock. Finally, we can get the CCSL constraint listed in Fig. 15(a), and the corresponding clock graph in Fig. 15(b).

4.3. Clock graph decomposition

The purpose of decomposition is to remove the *Union* and *SubClock* relations in a clock graph, and get sub-clock graphs for further analysis. The exact process is as follows.

- For the **Union** relation in Fig. 16(a), there are three situations. The Begin clock can tick together with C_1 or C_2 , or tick at the same time with C_1 and C_2 . Therefore, a *Union* relation like clock graph (I) can be decomposed into three sub-clock graphs, clock graphs (II), (III), and (IV) as illustrated in Fig. 16(a). This means we can transform the *Union* relation into two *Coincidence* relations, e.g., $Begin = C_1$ and $Begin = C_2$.

Algorithm 1: Clock graph composition algorithm.

Input: $CG_i = \{(CC_i, CA_i) | 1 \leq i \leq 2\}$: two clock graphs to be composed
Output: $CG = (CC, CA)$: the composed clock graph

```

1  $CC = null; CA = null;$ 
2 for  $\forall CG_i \in CGs$  do
    // compose all the nodes and edges of  $CG_1$  and  $CG_2$  directly
3    $CC = CC \cup \{CC_i\};$ 
4    $CA = CA \cup \{CA_i\};$ 
5 end
6  $k=1;$  // record the ID of Union
7 for  $\forall clk_i \in CG_1$  do
8   for  $\forall clk_j \in CG_2$  do
    // judge whether the two clocks are the same
9   if  $clk_i = clk_j$  then
    // if the same, delete original edges, add new nodes and edges
10     $CS_{out} = FindSuccessors(clk_i, CG);$  // successors of  $clk_i$  in  $CG$ 
11     $CE_{out} = FindSuccEdge(clk_i, CG);$  // constraints between  $clk_i$  and successors
12     $CS_{in} = FindPrecursors(clk_i, CG);$  // precursors of  $clk_i$  in  $CG$ 
13     $CE_{in} = FindPrecEdge(clk_i, CG);$  // constraints between  $clk_i$  and precursors
14    if  $|FindSuccessor| \geq 2$  or  $|FindPrecursor| \geq 2$  then
15       $U_k = newNode(Union);$  // add Union node
16       $CC = CC \cup \{U_k\};$ 
17      if  $|FindSuccessor| \geq 2$  then
18         $CA = CA \setminus FindSuccEdge;$  // delete original edges
19         $RUS = newUnionEdge(U_k, FindSuccessors);$  // create new Union edges
20         $RSS = newStricPreEdge(clk_i, U_k);$  // create StricPre edge
21      else
22         $CA = CA \setminus FindPrecEdge;$  // delete original edges
23         $RUS = FindPrecEdge(U_k, FindPrecursors);$  // create new Union edges
24         $RSS = newStricPreEdge(U_k, clk_i);$  // create StricPre edge
25       $CA = CA \cup \{RUS\} \cup \{RSS\};$  // add new edges
26       $k++;$ 
27    end
28 end
29 return  $CG;$ 

```

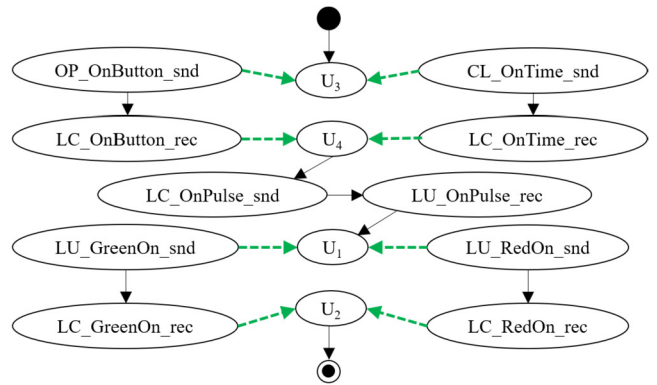
CCSL:

```

1. Begin1=OP_OnButton_snd+CL_OnTime_snd
2. OP_OnButton_snd < LC_OnButton_rec
3. CL_OnTime_snd < LC_OnTime_rec
4. End1=LC_OnButton_rec+LC_OnTime_rec
5. End1 < LC_OnPulse_snd
6. LC_OnPulse_snd < LU_OnPulse_rec
7. LU_OnPulse_rec < Begin
8. Begin=LU_RedOn_snd+LU_GreenOn_snd
9. LU_RedOn_snd < LC_RedOn_rec
10. LU_GreenOn_snd < LC_GreenOn_rec
11. End=LC_RedOn_rec+LC_GreenOn_rec

```

(a) CCSL constraints of composed sequence diagrams



(b) Clock graph of composed sequence diagrams

Fig. 15. Composed CCSL constraints and clock graph of the light controller system.

- For the **SubClock** relation $C_2 \subseteq C_1$ in 16(b), we can conclude with $C_1 < C_2$. This means that after C_1 is ticked, C_2 can be ticked or not. Therefore, a *SubClock* relation like clock graph (V) can be decomposed into two sub-clock graph, clock graphs (VI) and (VII) as shown in Fig. 16(b).

According to the above analysis, we design a decomposition algorithm. The input is a clock graph, and the output is a set of sub-clock graphs. By sub-clock graph, we mean that there is at least one path from the begin node to the end node. Firstly, we transform all edges in the input clock graph that have *Union* and *SubClock* relations to edges that only contain *StrictPre*, *Coincidence*, and *Exclude* relations. Then the depth first search algorithm is used to find all the atomic paths from the begin clock node to the end clock node, and these paths are transformed into multiple sets of clock graphs. The details are given in Algorithm 2. The complexity of this algorithm is $O(n^2)$, where n is the edge number of the input clock graph.

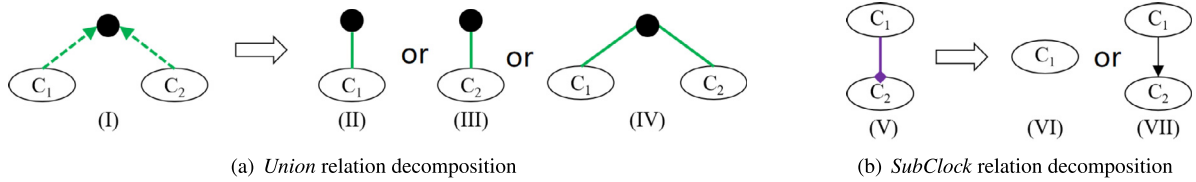


Fig. 16. Schematic diagram of the clock graph decomposition situations.

Algorithm 2: Clock graph decomposition algorithm.

Input: $CG = (CC, CA)$: the composed clock graph
Output: $SCGs = \{(CC_i, CA_i) | 1 \leq i \leq n\}$: a set of clock graphs

```

1 for  $\forall ca_i \in CA$  do
2   if  $ca_i.contains(Union)$  then
3     // deal with Union relations
4      $CA = CA \cup transformUnion(ca_i)$ ;
5      $CA = CA \setminus \{ca_i\}$ ;
6   else if  $ca_i.contains(SubClock)$  then
7     // deal with SubClock relations
8      $CA = CA \cup transformSubClock(ca_i)$ ;
9      $CA = CA \setminus \{ca_i\}$ ;
10  end
11 // find all the atomic sub-clock graphs
12  $SCG_n = getPath(CA, beginStr(CG), endStr(CG))$ ;
13  $SCGs = SCGs \cup \{SCG_n\}$ ;
14 for  $\forall cg_i \in SCGs$  do
15   for  $\forall cg_j \in SCGs$  do
16     // if two different clock graphs can be composed
17     if  $cg_i \neq cg_j \&\& haveUnion(cg_i, cg_j)$  then
18       // compose the clock graphs together
19        $SCG_i = composeCG(cg_i, cg_j)$ ;
20        $SCGs = SCGs \cup \{SCG_i\}$ ;
21     end
22   end
23 end
24 return  $CGs$ ;

```

We use the running example to illustrate the decomposition algorithm. Firstly the *Union* and *SubClock* relations in composed clock graph are transformed. The *Union* relation in the first row in Fig. 15(a) is transformed into the *Coincidence* relation between U_3 and $OP_OnButton_snd$ and CL_OnTime_snd . Through the depth-first search algorithm, we can obtain 4 paths from the start node B to the end node E , and then transform these paths into clock graphs. $OP_OnButton_snd$ and CL_OnTime_snd have a *Coincidence* relation, so these clock graphs can be composed to obtain some new clock graphs. After deleting the redundant clock graphs, the corresponding paths are shown in the Fig. 17. From the example, we could see there are 7 paths after decomposition. Among them, sub-clock graph SCG_1 and SCG_2 can be seen as decomposition result of the original clock graph in Fig. 13 which is transformed from the sequence diagram in Fig. 1(a). Similarly, sub-clock graph SCG_3 and SCG_5 are decomposition result of the clock graph transformed from the sequence diagram in Fig. 1(b). The other sub-clock graphs are created from the composition. It can be seen that the decomposition could reduce the complexity of the original big clock graph.

4.4. Graph based property and verification

From the SMT based verification results, we conclude two situations that are inconsistent in terms of clock graph:

- **hasCycle:** If a cycle of *StrictPre* relation is found in the clock graph, there will be inconsistent. Fig. 18(a) shows the schematic diagram of this situation. Suppose there are i clocks $C_1, C_2, \dots, C_i$, and a set of related clock constraints, $C_1 < C_2, C_2 < C_3, \dots, C_{i-1} < C_i$, and $C_i < C_1$, then there is a cycle between C_1 and C_i .
- **hasCoinExd:** If there are both *Coincidence* and *Exclude* relations between any two nodes in the clock graph, we consider that there is an inconsistency in the graph. Fig. 18(b) shows the schematic diagram of this situation. Suppose there are clocks, i.e., $C_1, C_2, \dots, C_i$. They have a *Coincidence* relation with each other, and there is an *Exclude* relation between C_i and C_j and a *Coincidence* relation between C_1 and C_j . Then it can be found that there are both *Exclude* and *Coincidence* relation between C_j and C_1 . In our running example, Path 7 in Fig. 17 shows a “hasCoinExd” problem. So the two sequence diagrams in Fig. 1 are inconsistent.

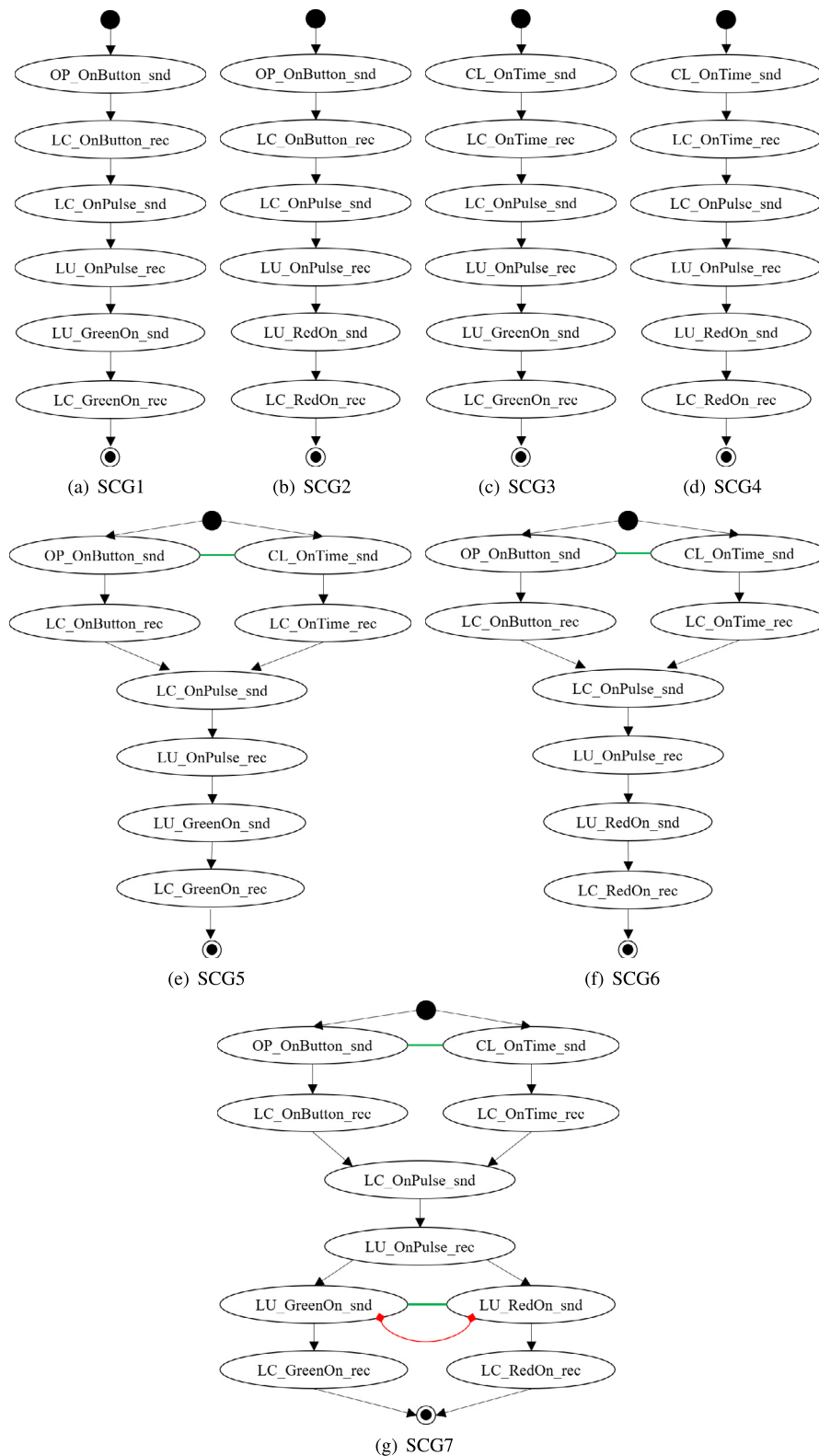


Fig. 17. Decomposed subclock graphs of light controller system.



Fig. 18. Two kinds of inconsistencies in terms of clock diagrams.

Table 2

Usage experiment results.

Case				SMT based Result	Clock graph based		
ID	Message	Fragment type	No.		hasCycle	hasCoinExd	FinalResult
1	11	alt	1	unsat	1 cycle	no	inconsistent
2	22	par	1	sat	no cycle	no	free
3	30	par	2	sat	no cycle	no	free
4	42	par	2	sat	no cycle	no	free
5	53	par+alt	2	unsat	no cycle	2 inconsistencies	inconsistent
6	60	par	2	sat	no cycle	no	free
7	72	par	2	sat	no cycle	no	free
8	90	par+alt	2	unsat	no cycle	no	free
9	102	par+alt	3	unsat	no cycle	4 inconsistencies	inconsistent
10	125	par	3	sat	no cycle	no	free

Proof of hasCycle inconsistency. For a set of clock constraints $C_1 < C_2$, $C_2 < C_3$, ..., $C_{i-1} < C_i$, and $C_i < C_1$, according to the definition of transitivity of *StricPre*, we have $\forall k \in N_{>0}, C_1[k] < C_i[k]$. Since $C_i < C_1$, that is $C_i[k] < C_1[k]$, there is a conflict between these two instant constraints in CCSL. Tracing these clocks back to the sequence diagram, they will be mapping to two events, one relation requires that C_i happens before C_1 , but another relation constrains that C_1 happens before C_i . This is event conflicts in sequence diagrams.

Proof of hasCoinExd inconsistency. For a set of clock constraints $C_1 = C_2$, $C_2 = C_3$, ..., $C_i = C_j$, and $C_j \# C_i$, according to the definition of transitivity of *Coincidence*, we have $\forall k \in N_{>0}, C_1[k] \equiv C_i[k]$, and $C_1[k] \equiv C_j[k]$. So we get $C_i[k] \equiv C_j[k]$. Since $C_i \# C_j$, that is $\forall k, l \in N_{>0}, C_i[k] \neq C_j[l]$, then we have $C_i[k] \neq C_j[k]$, there will be a conflict between these two instant constraints in CCSL. Tracing these clocks back to the sequence diagram, they will be mapping to two events, one relation requires that C_i should happen together with C_j , but another relation forbids C_i happens simultaneously with C_j . This is event conflicts in sequence diagrams.

These two proofs show that these two kinds of situations are inconsistent in terms of both sequence diagrams and CCSL constraints. So for each sub-clock graph, we could search it for the two kinds of inconsistencies using graph search algorithms. As long as there is *hasCycle* or *hasCoinExd* found in any sub-clock graph, the final verification result is inconsistent. It is to be noticed that all the sub-clock graphs pass the checking do not mean the final verification result is consistent. We can only say they are free of these two kinds of inconsistencies.

5. Evaluation

In this section, we evaluate both the SMT based method and the graph based method. The research questions are:

- What is the usage of each verification method? and
- How efficient are they?

We develop a tool *ConsistencySD* for supporting our methods. It is an online tool which could be accessed at <http://re4cps.org/consistentSD/>. In the evaluation, we conduct the experiments using a local version to reduce the network time influence. The machine is configured with CPU Intel (R) Core (TM) i5-5200U CPU @ 2.20GHz and Windows 10 (64bit).

5.1. Question 1: usage

We design 10 experiments from real railway cases to compare the verification results of both methods. For each experiment, we check the consistency of two sequence diagrams, transforming them into CCSL constraints, and checking them with two methods respectively. In SMT based verification, the bound is set 30, and the execution time is set to 10000s. In the process, we recorded the number of messages, the type and number of combined fragments in each experiment, the verification results of each method, the results of the *Cycle* inconsistency and *Coincidence with Exclude* inconsistency. Finally the results are shown in Table 2.

From the table, we can see that all the inconsistencies found at graph based method could be found in the SMT based verification, no matter the inconsistency type is “*hasCycle*” or “*hasCoinExd*”. It's the same the other way round. For the “*unsat*” in SMT based verification, the final results of clock graph based verification methods are also “*inconsistent*”. In

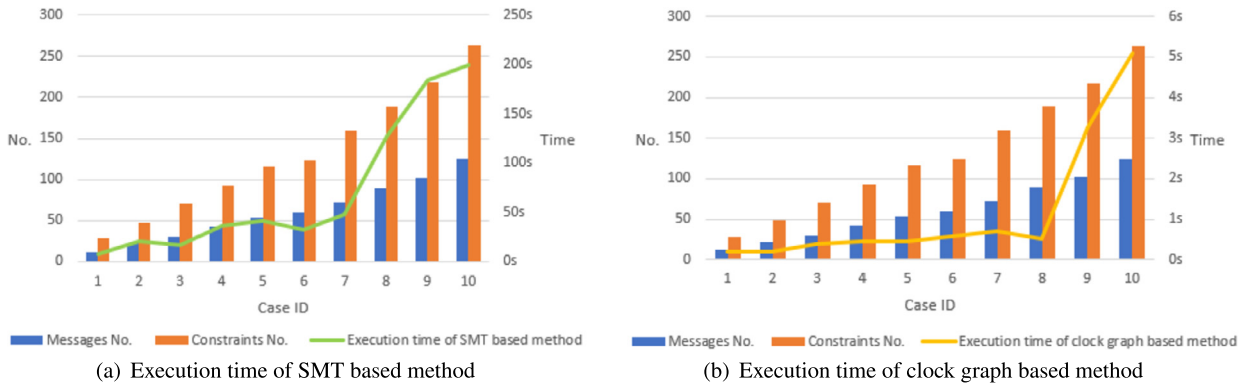


Fig. 19. Comparison of the execution time of our two methods.

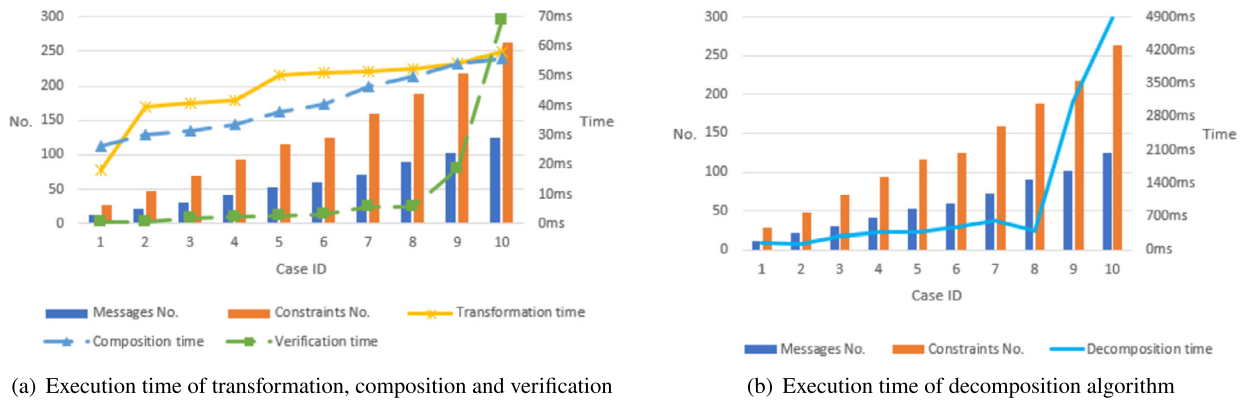


Fig. 20. Execution time of each algorithm.

theory, the SMT based verification method absolutely guarantees the correctness of the inconsistency verification result, but the clock graph based verification can only ensure the two kinds of inconsistencies defined beforehand. We depend on the SMT based method to discover new kinds of inconsistencies. Once a new kind of inconsistency is founded, we will check out whether it can be found in the clock graph, and redefine the clock graph based verification method to cover it.

5.2. Question 2: efficiency

To evaluate the verification efficiency of the methods in this paper, we choose 10 railway cases of different scales. The scale is measured by the number of messages and CCSL constraints. Among these cases, the message number ranges from 12 to 125, and the CCSL constraint number ranges from 28 to 263. With the same computer configuration, we carry out these 10 experiments. In the process, we record the number of messages, the number of constraints, and the execution time of both methods. For fairness, we execute each case 10 times, and compute the average execute time as the final result. Finally, the results are shown in Fig. 19.

From Fig. 19, we can see that the clock graph based verification method dramatically reduces the time cost. The execution time of the clock graph based verification method is less than 5.1s while the SMT based verification usually takes tens to hundreds of seconds. By comparison, the efficiency of the clock graph based verification method is increased by 36 to 241 times.

Moreover, in each method, the execution time is not linear to the number of messages or the constraints. It is already pointed out by Zhang et al. [20] that the time spent in SMT based verification is unpredictable. More constraints do not imply less run time, and less constraints do not imply more run time. From Fig. 19, it can be found that the execution time of the clock graph based method does not increase monotonically with the number of messages or the number of constraints. In order to find the reason, we investigate the execution time of each algorithm. There are 4 algorithms including CCSL transformation, clock graph composition, clock graph decomposition and clock graph based verification algorithm. We record the execution time of each algorithm in the above cases. Fig. 20 shows the relationship between the execution time of each phase and the number of messages and CCSL constraints. To reduce the influence of the operating system, each experiment is performed 10 times. We take the average time.

Through the analysis of Fig. 20, it can be found that the decomposition algorithm is the most time consuming. The figure curve of Fig. 20(b) is almost the same with Fig. 19(b). In fact, the execution time of decomposition is influenced by

the type and number of selected fragments in the sequence diagram. Case 7 has two *Parallel* combined fragments, which can be decomposed into 11 sub-clock graphs; Case 9 has two *Parallel* combined fragments and one *Alternative* combined fragment, which can be decomposed into 19 sub-clock graphs; while Case 8 only has one *Parallel* combined fragment and one *Alternative* combined fragment, which results only 7 sub-clock graphs.

Similarly, we analyze the execution time of the other three algorithms in Fig. 20(a). They are monotonically increasing. The execution time of the transformation algorithm is generally increasing, which may be related to the type of messages in the sequence diagrams. The clock graph composition algorithm linearly increases with the number of CCSL constraints, which is consistent with the time complexity analyzed before. The execution time of the verification algorithm also increases with the number of constraints. The sharp increase in the last two experiments is mainly related to two types of relations in the sequence diagrams, parallel and negative. Parallel is transformed to *Union*, and Negative is *Subclock*. The processing time of a *Union* relation will increase by 0.5 times, and the processing time of a *SubClock* relation will increase by 1 time. There is at most one Parallel combined fragment in the first 8 cases, while there are two Parallel combined fragments in Case 9 and three Parallel combined fragments in Case 10. The verification time of the latter two experiments is significantly increased.

To sum up, we conclude that the SMT based verification method can fully guarantee the accuracy of the inconsistency verification result, and the graph based verification method can greatly improve the verification efficiency. The two kinds of consistencies covered by the clock graph based method are found out in real transit system documents through collaboration with a Chinese railway company. In real case verifications, we suggest to combine these two methods to verify the consistency of safety requirements. Firstly graph based verification method is used to quickly verify the consistency of requirements. Inconsistent result could be directly fed back to the requirements engineer. For consistent results, we use the SMT based method to further the inconsistency verification. If the result is unsat, then inconsistent is fed back to the requirements engineer; otherwise, it means that these requirements are consistent. Moreover, when the new kind of inconsistency is found by the SMT based method, we could like to check out whether it could be found on the clock graph. We could define new properties on the graph, and try to extend the verification scope of the clock graph based method.

6. Related work

There are many researches on formalizing sequence diagram 2.0 in various formalism. For example, Lima et al. create a PROMELA-based model from UML interactions expressed in sequence diagrams, and use SPIN model checker to simulate the execution and to verify properties written in Linear Temporal Logic (LTL) [4]. Han et al. realize a full-map from sequence diagram to timed automata network [5]. Formal verification is carried out using automated model checkers like UPPAAL. Tu et al. establish formal specification rules for UML 2.0 diagrams based on CCS, calculus of communicating system. Theorems are developed to obtain the formal specification of UML 2.0 sequence diagrams with loop, alternative flow and concurrency [6]. Li et al. define a static semantics for UML sequence diagrams to support checking the well-formedness of a sequence diagram in the context of other diagrams [21], i.e. its consistency with a class diagram and a state diagram. Shen et al. present a template semantics, for describing the operational semantics of behavioral notations, especially the combined fragments and other constructs of sequence diagrams [22]. Muram et al. translate sequence diagrams into temporal logic based constraints (LTL) and formal behavior specifications (i.e., symbolic model language (SMV)), respectively [23]. The model checker NuSMV is used to verify the containment relationship.

Various petri nets are also used. For example, [7] defines and explains the relationship between sequence diagrams and normal Petri nets. Cunha et al. also translate UML sequence diagrams to Petri nets and verify deadlock freeness, reachability, safety and liveness properties by using a model checker [24]. Soares et al. present an approach to automatically translate Sequence Diagrams to colored Petri nets (CPN) ready for execution with CPN Tools [8]. Bowles et al. also transform UML 2 sequence diagrams to CPNs [25]. Formal transformation rules are extended to consider modelling aspects such as stochastic and real-time behavior. Existing Petri net analysis and verification tools are exploited. [9] transforms sequence diagrams into equivalent Queuing Petri Nets. Paper [10] proposes an automatic translation of Statecharts and Sequence Diagrams into Generalized Stochastic Petri Nets, and a composition of the resulting net models. In the paper [26] formal semantics is introduced for many concepts of sequence diagrams by means of colored high-level Petri nets as a formal model. Petri net fragments are then connected by composition operators according to the enclosing combined fragments operations. The semantics is similar to that in [27].

However the above works do not deal with negative operator which is important for safety requirements while our approach addresses that. Moreover petri net is mostly used. But petri nets are too expressive to be verified efficiently. Our CCSL when transformed to clock graph is a simplified version of petri net [19]. Also CCSL provides various kinds of analysis tools such as safety analysis in [19] although in this paper we only focus on consistency analysis using the state-of-the-art model checker Z3. Moreover, we provide a clock graph based method, a lightweight formal method for very high efficiency.

This paper is mostly related to Dhaou et al.'s work [28]. It extends an existing causal semantics to deal with sequence diagrams with the most popular combined fragments (Alt, Opt and Loop), by processing the sequence diagram as a whole. They have proposed several rules to derive the partial orders between the events. But it does not support efficient verification. CCSL constraints also describe event relations, and even provide more than partial orders.

For more efficient verification, there are many studies on the compositional verification of sequence diagrams. In [29], a compositional semantics of sequence diagrams is defined based on labelled event structures, and an automated technique

is presented based on Alloy that relies on the true-concurrent semantics. Yu et al. propose an event-based formal semantics for sequence diagrams and present a new functional decomposition approach based on sequence diagrams [30]. The new decomposition idea is from a global system to global subsystems. Under the decomposition method global subsystems can be recomposed into the global system using the composition operation called union without necessarily considering assumptions such as communication. Widl et al. present an approach for composing concurrently evolved sequence diagrams in accordance to the behavior given in state machine models [31]. They describe the problem of merging sequence diagrams formally using SAT solvers. However, similarly to [32], the approach does not merge complex sequence diagrams. Araújo et al. propose a further approach to composition of sequence diagrams by composing sequence diagram operators directly [33]. In fact, our clock graph based method is also a kind of compositional verification. It first decomposes the clock graph, and uses the “and” verification results of each sub-clock graphs. Compared with above compositional verification, our method provides automated decomposition, and supports graph based verification.

7. Conclusion

In this paper, we propose a consistency verification for safety requirements in terms of sequence diagrams. Our paper makes the following contributions: (1) the formalization of safety requirements in terms of sequence diagram is realized by CCSL constraints; (2) the consistency property of safety requirements in terms of CCSL constraints is checked by SMT based tool MyCCSL; and (3) a clock graph is defined and a clock graph based consistency verification method is defined for improving efficiency.

In the future, we want to improve our work from the following two aspects. Firstly, counterexamples returned from inconsistent cases will be used to explain the reasons of inconsistency, and provide strategies to refine the safety requirements in terms of sequence diagrams. Secondly, we will apply our methods to more real cases, and try to discover and define new inconsistency properties for extending the verification scope of the clock graph based method.

CRedit authorship contribution statement

Xiaohong Chen: Writing original draft, Review and editing, Idea of SMT based verification. **Qianqian Liu:** Software, Experiments, Visualization, Writing, Review and Editing. **Frédéric Mallet:** Idea of clock graph based verification, Writing, Review and Editing. **Qin Li:** Idea of composition, Review and Editing. **Shubin Cai:** Conceptualization, Methodology, Validation, Review and Editing. **Zhi Jin:** Idea of decomposition, Review and Editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This research was supported by the National Natural Science Foundation of China (62192731), Shanghai Science and Technology Committee(STCSM 20DZ1100300, 20ZR1416000), and the French National Research Agency SIM ANR-19-CE25-0008-02. The author Qin Li is supported by the foundation of Shenzhen Institute of Artificial Intelligence and Robotics for Society (AC01202005020).

References

- [1] Z. Jin, Environment Modeling-Based Requirements Engineering for Software Intensive Systems, Morgan Kaufmann, Massachusetts, USA, 2018.
- [2] CENELEC, 50129: Railway application-Communications, signaling and processing systems-Safety related electronic systems for signaling, European Committee for Electrotechnical Standardization, 2018.
- [3] N.G. Leveson, Software safety in computer-controlled systems, *IEEE Comput.* 17 (1984) 48–55.
- [4] V. Lima, C. Talhi, D. Mouheb, M. Debbabi, L. Wang, M. Pourzandi, Formal verification and validation of UML 2.0 sequence diagrams using source and destination of messages, *Electron. Notes Theor. Comput. Sci.* 254 (2009) 143–160.
- [5] D. Han, J. Xing, Q. Yang, H. Wang, X. Zhang, Formal sequence: extending UML sequence diagram for behavior description and formal verification, in: 40th IEEE Annual Computer Software and Applications Conference, COMPSAC Workshops 2016, Atlanta, GA, USA, June 10–14, 2016, 2016, pp. 474–481.
- [6] T. Peng, G. Ding, Formal specification and automated verification of uml2. 0 sequence diagrams, in: 2012 IEEE International Conference on Granular Computing, 2012, pp. 370–375.
- [7] T.S. Staines, Transforming uml sequence diagrams into petri net, *J. Comput. Commun.* 10 (2013) 72–81.
- [8] J.A.C. Soares, B. Lima, J.P. Faria, Automatic model transformation from UML sequence diagrams to coloured petri nets, in: Proceedings of the 6th International Conference on Model-Driven Engineering and Software Development, MODELSWARD 2018, Funchal, Madeira - Portugal, January 22–24, 2018, 2018, pp. 668–679.
- [9] V.V. Doc, H.Q. Thang, N.T. Bach, Development of the rules for transformation of UML sequence diagrams into queueing petri nets, in: Industrial Networks and Intelligent Systems - 4th EAI International Conference, INISCOM 2018, Da Nang, Vietnam, August 27–28, 2018, Proceedings, 2018, pp. 122–144.
- [10] S. Bernardi, S. Donatelli, J. Merseguer, From uml sequence diagrams and statecharts to analysable petri net models, in: Proceedings of the 3rd International Workshop on Software and Performance, 2002, pp. 35–45.

- [11] F. Mallet, Clock constraint specification language: specifying clock constraints with UML/MARTE, *ISSE 4* (2008) 309–314.
- [12] OMG, Uml profile for modelling and analysis of real-time and embedded systems (marTE), <http://www.omgmarTE.org/>, 2011.
- [13] X. Chen, F. Mallet, X. Liu, Formally verifying sequence diagrams for safety critical systems, in: *International Symposium on Theoretical Aspects of Software Engineering*, 2020, pp. 217–224.
- [14] M. Zhang, F. Dai, F. Mallet, Periodic scheduling for MARTE/CCSL: theory and practice, *Sci. Comput. Program.* 154 (2018) 42–60.
- [15] C. Andre, Syntax and Semantics of the Clock Constraint Specification Language (CCSL), Research Report, INRIA, 2009.
- [16] L.M. de Moura, N. Bjørner, Z3: an efficient SMT solver, in: *Tools and Algorithms for the Construction and Analysis of Systems*, 14th International Conference, TACAS 2008, 2008, pp. 337–340.
- [17] Object Management Group, UML 2.4.1 superstructure specification, 2018-11-16, <http://www.omg.org/spec/UML/2.4.1>.
- [18] L. Yin, J. Liu, Z. Ding, F. Mallet, R. de Simone, Schedulability analysis with ccsL specifications, in: *APSEC*, 2013, pp. 414–421.
- [19] F. Mallet, J. Millo, R. de Simone, Safe CCSL specifications and marked graphs, in: *11th ACM/IEEE International Conference on Formal Methods and Models for Codesign, MEMCODE 2013, Portland, OR, USA, October 18–20, 2013*, 2013, pp. 157–166.
- [20] M. Zhang, F. Song, F. Mallet, X. Chen, Smt-based bounded schedulability analysis of the clock constraint specification language, in: *Fundamental Approaches to Software Engineering (FASE'19)*, in: LNCS, vol. 11424, Springer, 2019, pp. 61–78.
- [21] X. Li, Z. Liu, J. He, A formal semantics of UML sequence diagram, in: *15th Australian Software Engineering Conference (ASWEC 2004)*, Melbourne, Australia, 13–16 April 2004, 2004, pp. 168–177.
- [22] H. Shen, A. Virani, J. Niu, Formalize UML 2 sequence diagrams, in: *11th IEEE High Assurance Systems Engineering Symposium, HASE 2008, Nanjing, China, December 3–5, 2008*, 2008, pp. 437–440.
- [23] F.U. Muram, H. Tran, U. Zdun, A model checking based approach for containment checking of UML sequence diagrams, in: *23rd Asia-Pacific Software Engineering Conference, APSEC 2016, Hamilton, New Zealand, December 6–9, 2016*, 2016, pp. 73–80.
- [24] E. Cunha, M. Custódio, H. Rocha, R.S. Barreto, Formal verification of UML sequence diagrams in the embedded systems context, in: *Brazilian Symposium on Computing System Engineering, SBESC 2011, Florianopolis, Brazil, November 7–11, 2011*, 2011, pp. 39–45.
- [25] J. Bowles, D.A. Meedeniya, Formal transformation from sequence diagrams to coloured petri nets, in: *17th Asia Pacific Software Engineering Conference, APSEC 2010, Sydney, Australia, November 30 - December 3, 2010*, 2010, pp. 216–225.
- [26] C. Eichner, H. Fleischhack, R. Meyer, U. Schrimpf, C. Stehno, Compositional semantics for UML 2.0 sequence diagrams using petri nets, in: *12th International SDL Forum*, 2005, pp. 133–148.
- [27] J. Fernandes, S. Tjell, J. Jorgensen, O. Ribeiro, Designing tool support for translating use cases and uml 2.0 sequence diagrams into a coloured petri net, in: *International Conference on Scenarios and State Machines*, 2007, pp. 2–12.
- [28] F. Dhaoui, I. Mouakher, J.C. Attiogbé, K. Bsaies, A causal semantics for UML2.0 sequence diagrams with nested combined fragments, in: *ENASE 2017 - Proceedings of the 12th International Conference on Evaluation of Novel Approaches to Software Engineering*, Porto, Portugal, April 28–29, 2017, 2017, pp. 47–56.
- [29] M. Alwanain, B. Bordbar, J.K.F. Bowles, Automated composition of sequence diagrams via alloy, in: *IEEE International Conference on ModelDriven Engineering and Software Development 23*, 2015, pp. 384–391.
- [30] X. Yu, J.-M. Jiang, Z. Hong, Decomposition and composition of sequence diagrams, in: *12th International Symposium on Theoretical Aspects of Software Engineering*, 2018, pp. 76–83.
- [31] M. Widl, A. Biere, P. Brosch, U. Egly, M. Heule, G. Kappel, M. Seidl, H. Tompits, Guided merging of sequence diagrams, in: *SLE 2012*, in: LNCS, vol. 7745, Springer, 2013, pp. 164–183.
- [32] H. Liang, Z. Diskin, J. Dingel, E. Posse, A general approach for scenario integration, in: *MoDELS'08*, in: LNCS, vol. 5301, Springer, 2008, pp. 204–218.
- [33] J. Araújo, J. Whittle, D. Kim, Modeling and composing scenario-based requirements with aspects, in: *RE 2004*, IEEE, 2004, pp. 58–67.